8 Kasım 2026

BM201
İleri Programlama
Project Proposal Template

Proje Ekibi:	Sinan Gün, 25118080047, 1 Hasan Can Bayındır, 23118080091, 1 İsmail Can Anlaş, 24118080098, 1 Ahmet Kıymık, 24118080704, 1
Proje İsmi:	**CezAI (Yapay Zeka Destekli Otoyol Hız Takip ve Dinamik Ceza Yönetim Sistemi)**
Proje Konusu ve Amacı:	**Proje Konusu ve Amacı:** CezAI, otoyollarda akan araç trafiğini yüksek çözünürlüklü kamera akışları üzerinden gerçek zamanlı bilgisayarlı görü ve derin öğrenme (Computer Vision / Deep Learning) modelleriyle takip eden, araçların anlık hızlarını piksel-metre perspektif kalibrasyonuyla (Homography) tespit eden, yasal hız limitini aşan araçlara aşım oranına göre kademeli ceza uygulayan ve tüm denetim verilerini ilişkisel veritabanında saklayan akıllı bir trafik denetleme ve yönetim sistemidir. Geleneksel radar ve hız tespit sistemleri yüksek donanım maliyetine sahip olup yalnızca tek bir noktadan anlık ölçüm yapabilmektedir. CezAI, mevcut otoyol ve MOBESE kameralarını yapay zeka algoritmaları ile entegre ederek ek sensör maliyetine gerek kalmaksızın araçları video akışı boyunca dinamik olarak takip eder. Sistem, araç tipine (otomobil, kamyon, otobüs, motosiklet) göre değişen yasal hız sınırlarını otomatik uygular, Karayolları Trafik Kanunu baremlerine göre aşım yüzdesini hesaplar ve trafik ekiplerinin kullanımına hazır dijital ceza tutanakları üretir. Sistemde iki farklı kullanıcı tipi bulunacaktır: Sistem Yöneticisi (Admin) ve Trafik Operatörü (Operator). Yönetici; sisteme yeni kamera kaynaklarını ve video akışlarını bağlayabilecek, otoyol hız sınırı parametrelerini (araç sınıfı bazlı limitler, hava koşulu çarpanları, tolerans oranları) güncelleyebilecek ve kullanıcı hesaplarını yönetebilecektir. Trafik Operatörü ise sisteme giriş yaparak canlı kamera kokpitini izleyebilecek, hız sınırını aşan araçları kırmızı uyarı çerçeveleriyle anlık görebilecek, plakaya veya ihlal derecesine göre filtreleme yapabilecek ve onaylanan ihlalleri resmi denetim raporu (PDF/Excel) olarak sistemden dışa aktarabilecektir. Sistemde araçlara ve ihlallere dair şu veriler tutulacaktır: Araç Takip ID'si, Plaka Bilgisi, Araç Türü (Otomobil, Kamyon, Otobüs, Motosiklet), Ölçülen Anlık Hız (km/s), Geçerli Hız Sınırı (km/s), Hız Aşım Yüzdesi (%), İhlal Seviyesi (1. Kademe: %10-%30, 2. Kademe: %30-%50, 3. Kademe: %50 üzeri), Hesaplanan Para Cezası Tutarı, Ceza Puanı, İhlal Zamanı (Timestamp), Kamera Lokasyonu ve İhlal Anı Kanıt Fotoğrafı (Snapshot). Giriş

	ve yetkilendirme ekranında kullanıcılar şifrelenmiş kimlik bilgileriyle oturum açacak ve yetki seviyelerine göre ilgili yönetim paneline yönlendirilecektir. **OOP tarafında projeyi şu şekilde yapılandırmayı planlıyoruz:** • **Encapsulation (Kapsülleme):** Sınıflardaki kritik değişkenler (ölçülen hız, plaka bilgisi, ceza tutarı, koordinatlar) private / protected olarak kapsüllenecektir. Dış dünyadan erişim kontrollü özellikler (@property, getter/setter metotları) ve iş kuralı doğrulayıcıları üzerinden sağlanacaktır. Hatalı veri girişleri veya sınır dışı değer atamaları özel istisnalar (exception) fırlatılarak engellenecektir. • **Inheritance (Kalıtım):** Vehicle adında soyut bir temel sınıf (abstract class) tanımlanacaktır. Car, Truck, Bus ve Motorcycle sınıfları bu sınıftan türeyecektir. Benzer şekilde kullanıcı hiyerarşisinde User soyut sınıfından Admin ve Operator sınıfları miras alacaktır. Böylece ortak özellikler (plaka, hız, yön, koordinat, kimlik bilgileri) üst sınıfta tek merkezde toplanacak, araç tipine özel nitelikler alt sınıflarda genişletilecektir. • **Polymorphism (Çok Biçimlilik):** Farklı araç sınıfları get_speed_limit() ve calculate_excess_ratio() gibi ortak metotları kendi yasal kurallarına göre ezecektir (override). Örneğin otoyolda Car için limit 140 km/s iken, Truck için 90 km/s olarak polimorfik işletilecektir. Ayrıca ceza hesaplama motoru IPenaltyStrategy arayüzünü uygulayan sınıflar üzerinden çalışma zamanında dinamik olarak çağrılacak; ihlal arama ve filtreleme işlevleri farklı sorgu parametreleriyle (plakaya göre, araç sınıfına göre, tarih aralığına göre) esnek biçimde çalıştırılabilecektir. • **Abstraction & Tasarım Kalıpları:** Sistem bileşenleri soyut sınıflar (ABC) ve arayüzler (Interface) ile birbirinden bağımsızlaştırılacaktır. Kademeli ceza hesabı için Strategy Pattern, ihlal algılandığında veritabanı kaydı ve görsel alarm tetiklemek için Observer Pattern, tespit edilen sınıfa göre uygun araç nesnesini üretmek için Factory Pattern ve veri erişimi için Repository Pattern kullanılacaktır. **Teknik olarak:** • Masaüstü uygulaması olarak Python 3.11+ dilinde geliştirilecektir (Yapay zeka ve bilgisayarlı görü kütüphaneleriyle yerel ve yüksek performanslı entegrasyon). • Bilgisayarlı görü ve takip için OpenCV (video yakalama, homografi perspektif düzeltme), Ultralytics YOLOv8 (araç tespiti ve sınıflandırma) ve ByteTrack (kareler arası sürekli nesne takibi) kullanılacaktır. • İlişkisel veritabanı (SQLite / PostgreSQL) ve kurumsal ORM olarak SQLAlchemy kullanılacak; araç logları, ihlal kayıtları ve kullanıcı hesapları burada tutulacaktır. • Kullanıcı arayüzü olarak modern, ergonomik ve duyarlı PyQt6 (Qt Designer) masaüstü kontrol paneli geliştirilecektir. • Hata yönetimi için özel istisna sınıfları (SpeedCalculationException, StreamConnectionError vb.)

	ve yapılandırılmış try-except blokları kullanılacak, kullanıcıya anlamlı hata ve durum mesajları gösterilecektir. • Proje kodları GitHub üzerinde versiyonlanacak, modüler paket yapısında tutulacak ve pytest ile birim testleri uygulanacaktır. **Planlanan sınıflar:** • **Vehicle (abstract / ABC) —** Tüm araçların ortak özelliklerini (plaka, takip ID, piksel koordinatları, anlık hız) ve soyut metotlarını tanımlayan temel sınıf. • **Car (extends Vehicle) —** Otomobil sınıfı (otoyol hız limiti: 140 km/s, binek araç ceza katsayısı). • **Truck (extends Vehicle) —** Kamyon ve ağır vasıta sınıfı (otoyol hız limiti: 90 km/s, ağır vasıta güvenlik katsayısı). • **Bus (extends Vehicle) —** Yolcu otobüsü sınıfı (otoyol hız limiti: 100 km/s, ticari taşıma hız toleransı). • **Motorcycle (extends Vehicle) —** Motosiklet sınıfı (otoyol hız limiti: 100 km/s, iki tekerlekli taşıt dinamikleri). • **ViolationRecord —** Hız ihlali anının detaylarını (tarih/saat, tespit hızı, yasal limit, aşım yüzdesi, para cezası, ceza puanı, görsel kanıt dosya yolu) saklayan model sınıfı. • **User (abstract) —** Sistem kullanıcılarının ortak kimlik ve yetki özelliklerini tanımlayan soyut sınıf. • **Admin (extends User) —** Kamera bağlantıları, hız sınırları ve kullanıcı yönetimi yetkilerine sahip yönetici sınıfı. • **Operator (extends User) —** Canlı radar akışını izleyen, ihlalleri doğrulayan ve ceza tutanağı raporlayan denetim memuru sınıfı. • **IPenaltyStrategy (interface) —** Aşım yüzdesine göre kademeli ceza ve ceza puanı hesaplama kurallarını tanımlayan strateji arayüzü. • **HighwayPenaltyStrategy (implements IPenaltyStrategy) —** 2918 sayılı Karayolları Trafik Kanunu baremlerini (%10-%30, %30-%50, %50+) dinamik hesaplayan strateji sınıfı. • **IViolationObserver (interface) —** İhlal anında tetiklenecek olay dinleyici arayüzü. • **DatabaseLoggerObserver (implements IViolationObserver) —** İhlal oluştuğunda SQLAlchemy ORM üzerinden anında veritabanına kayıt işleyen gözlemci. • **UIAlarmObserver (implements IViolationObserver) —** İhlal anında canlı operatör kokpitinde sesli ve görsel alarm üreten gözlemci. • **VehicleFactory —** YOLOv8 tespit sonuçlarından elde edilen sınıf etiketine göre ilgili Vehicle türevini üreten fabrika sınıfı. • **IViolationRepository & SqlAlchemyViolationRepository —** Veritabanı CRUD operasyonlarını iş kurallarından soyutlayan depo sınıfı.